

1. Fuzzy Logic Controller – Motor Speed Control

Algorithm: ## 1. Define the speed error as the input variable and control signal as the output variable. ## 2. Define membership functions for the speed error as Negative, Zero, and Positive. ## 3. Define membership functions for the control signal as Low, Medium, and High. ## 4. Create fuzzy rules to relate the speed error to the control signal. ## 5. Give different error values as inputs and compute the fuzzy control output. ## 6. Display the control output for each error value and stop the program.

Program: ## import numpy as np ## import skfuzzy as fuzz ## from skfuzzy import control as ctrl ## error = ctrl.Antecedent(np.arange(-10, 11, 1), 'error') ## control = ctrl.Consequent(np.arange(0, 101, 1), 'control') ## error['negative'] = fuzz.trimf(error.universe, [-10, -10, 0]) ## error['zero'] = fuzz.trimf(error.universe, [-5, 0, 5]) ## error['positive'] = fuzz.trimf(error.universe, [0, 10, 10]) ## control['low'] = fuzz.trimf(control.universe, [0, 0, 50]) ## control['medium'] = fuzz.trimf(control.universe, [25, 50, 75]) ## control['high'] = fuzz.trimf(control.universe, [50, 100, 100]) ## rule1 = ctrl.Rule(error['negative'], control['low']) ## rule2 = ctrl.Rule(error['zero'], control['medium']) ## rule3 = ctrl.Rule(error['positive'], control['high']) ## system = ctrl.ControlSystem([rule1, rule2, rule3]) ## simulation = ctrl.ControlSystemSimulation(system) ## print("Error\tControl Output") ## for e in [-8, -3, 0, 4, 8]: ##simulation.input['error'] = e ##simulation.compute() ##print(e, "\t", round(simulation.output['control'], 2))

2. ANN – Learn $y = x^2$

Algorithm: ## 1. Define suitable training input values and calculate the corresponding output values using $y = x^2$. ## 2. Create an ANN model with an input layer, hidden layer, and output layer. ## 3. Train the ANN using the training input and output data. ## 4. Provide a new input value to the trained ANN. ## 5. Predict the output for the new input. ## 6. Display the predicted result and stop the program.

Program: ## import numpy as np ## from sklearn.neural_network import MLPRegressor ## X_train = np.array([[1], [2], [3], [4], [5]]) ## y_train = np.array([1, 4, 9, 16, 25]) ## model = MLPRegressor(hidden_layer_sizes=(10,), max_iter=5000, random_state=1) ## model.fit(X_train, y_train) ## X_test = np.array([[6]]) ## predicted = model.predict(X_test) ## print("Input:", X_test[0][0]) ## print("Actual:", 36) ## print("Predicted:", round(predicted[0], 2))

3. PID Controller – First-Order System

Algorithm: ## 1. Define the desired setpoint and initialize the system output. ## 2. Set the proportional, integral, and derivative gains K_p , K_i , and K_d . ## 3. Calculate the error between the setpoint and actual output at every time step. ## 4. Calculate the proportional, integral, and derivative terms. ## 5. Calculate the PID control signal and update the first-order system output. ## 6. Repeat the process for several time steps and display the step response.

Program: ## Kp = 2.0 ## Ki = 0.1 ## Kd = 0.5 ## setpoint = 1 ## output = 0 ## integral = 0 ## previous_error = 0 ## print("Time\tOutput\tControl") ## for t in range(1, 11): ##error = setpoint - output ##integral = integral + error ##derivative = error - previous_error ##control = Kp * error + Ki * integral + Kd * derivative ##output = output + 0.1 * (control - output) ##print(t, "\t", round(output, 2), "\t", round(control, 2)) ##previous_error = error

4. Adaptive Controller – $y = 2x$

Algorithm: ## 1. Define the desired relationship $y = 2x$ and initialize the control parameter. ## 2. Provide an input value to the system and calculate the actual output. ## 3. Calculate the error between the desired output and actual output. ## 4. Update the control parameter according to the output error. ## 5. Repeat the process for different input values until the parameter approaches the required value. ## 6. Display the final adaptive parameter and output.

Program: ## learning_rate = 0.01 ## K = 1.0 ## print("Input\tOutput\tError\tGain") ## for x in range(1, 11): ##desired = 2 * x ##output = K * x ##error = desired - output ##K = K + learning_rate * error / x ##output = K * x ##print(x, "\t", round(output, 2), "\t", round(error, 2), "\t", round(K, 2))

5. Genetic Algorithm – Minimum of $f(x) = x^2$

Algorithm: ## 1. Define the objective function $f(x) = x^2$. ## 2. Generate random candidate solutions within the specified range. ## 3. Evaluate the fitness of each candidate using the objective function. ## 4. Select the candidate with the minimum fitness value. ## 5. Apply mutation to generate new candidate

solutions. ## 6. Repeat the selection and mutation process for several generations and display the best solution.

```
## Program: ## import random ## def fitness(x): ##return x ** 2 ## x = random.uniform(-10, 10) ##
print("Generation\tX\tFitness") ## for generation in range(1, 11): ##new_x = x + random.uniform(-1, 1) ##if
fitness(new_x) < fitness(x): ##x = new_x ##print(generation, "\t\t", round(x, 2), "\t", round(fitness(x), 4)) ##
print("\nBest Solution =", round(x, 2)) ## print("Minimum Value =", round(fitness(x), 4))
```

6. Q-Learning – 3×3 Grid

Algorithm: ## 1. Define the 3×3 grid environment with the start state and goal state. ## 2. Initialize the Q-table with zeros and set the discount factor. ## 3. Select a random starting state and choose a valid action. ## 4. Update the Q-value using the reward and maximum future Q-value. ## 5. Continue moving between states until the goal state is reached. ## 6. Repeat the training process for several episodes and display the learned Q-table and optimal path.

```
## Program: ## import numpy as np ## import random ## R = np.array([[ -1, 0, -1, -1, -1, -1, -1, -1, -1], ##
[0, -1, 0, -1, -1, -1, -1, -1, -1], ##[-1, 0, -1, -1, -1, 0, -1, -1, -1], [-1, -1, -1, -1, 0, -1, 0, -1, -1], [-1, -1, -1, 0, -1,
0, -1, 0, -1], [-1, -1, 0, -1, 0, -1, -1, -1, 100], [-1, -1, -1, 0, -1, -1, -1, 0, -1], [-1, -1, -1, -1, 0, -1, 0, -1, 100],
[-1, -1, -1, -1, -1, 0, -1, 0, -1]]) ## Q = np.zeros((9, 9)) ## gamma = 0.8 ## for episode in range(100): ##
state = 0 ##while state != 8: ##actions = np.where(R[state] >= 0)[0] ## action =
random.choice(actions) ##Q[state, action] = R[state, action] + gamma * np.max(Q[action]) ##state =
action ## print("Q Table:") ## print(np.round(Q, 2)) ## state = 0 ## print("\nOptimal Path:") ## while state
!= 8: ##print(state, end=" -> ") ##state = np.argmax(Q[state]) ## print(8)
```

7. MPC – First-Order System

Algorithm: ## 1. Define the desired output value as 1 and initialize the system output to zero. ## 2. Calculate the error between the desired output and current output. ## 3. Calculate the MPC control action based on the current error. ## 4. Apply the control action to the first-order system and update the output. ## 5. Repeat the prediction and control process for every time step. ## 6. Display the output and control action until the system reaches the desired value.

```
## Program: ## setpoint = 1 ## output = 0 ## print("Step\tOutput\tControl") ## for step in range(1, 11):
##error = setpoint - output ##control = 0.5 * error ##output = output + control ##print(step, "\t",
round(output, 2), "\t", round(control, 2))
```

8. A* – 5×5 Grid with Obstacles

Algorithm: ## 1. Define the 5×5 grid with free cells and obstacle cells, and define the start and goal positions. ## 2. Initialize the open list with the start node and an empty closed list. ## 3. Calculate the cost function $f = g + h$ using the path cost and Manhattan distance heuristic. ## 4. Select the node with the lowest f value from the open list. ## 5. Generate valid neighboring cells and update their costs and parent nodes. ## 6. Continue until the goal is reached, reconstruct the shortest path, and display it.

```
## Program: ## import heapq ## grid = [[0,0,0,0,0], [0,1,1,1,0], [0,0,0,1,0], [0,1,0,0,0], [0,0,0,0,0]] ## start
= (0,0) ## goal = (4,4) ## def heuristic(a, b): ## return abs(a[0]-b[0]) + abs(a[1]-b[1]) ## open_list = [(0,
start)] ## cost = {start: 0} ## parent = {start: None} ## while open_list: ## _, current =
heapq.heappop(open_list) ## if current == goal: ## break ##for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
##neighbor = (current[0]+dx, current[1]+dy) ##if 0 <= neighbor[0] < 5 and 0 <= neighbor[1] < 5 and
grid[neighbor[0]][neighbor[1]] == 0: ## new_cost = cost[current] + 1 ##if neighbor not in cost or new_cost
< cost[neighbor]: ##cost[neighbor] = new_cost ##priority = new_cost + heuristic(neighbor, goal) ##
heapq.heappush(open_list, (priority, neighbor)) ##parent[neighbor] = current ## path = [] ## current =
goal ## while current is not None: ##path.append(current) ##current = parent.get(current) ##
path.reverse() ## print("Shortest Path:", path) ## print("Path Cost:", cost[goal])
```

9. PSO – Minimum of $f(x) = (x - 3)^2$

Algorithm: ## 1. Define the objective function $f(x) = (x - 3)^2$. ## 2. Initialize particles with random positions and velocities. ## 3. Calculate the fitness of each particle and store its personal best position. ## 4. Determine the global best position from all personal best positions. ## 5. Update particle velocities and positions using personal best and global best values. ## 6. Repeat the process for several iterations and display the optimal position and minimum fitness.

```
## Program: ## import random ## def fitness(x): ##return (x - 3) ** 2 ## particles = [random.uniform(0, 10)
for _ in range(5)] ## velocity = [0] * 5 ## pbest = particles[:] ## gbest = min(pbest, key=fitness) ##
print("Iteration\tGlobal Best\tFitness") ## for i in range(1, 11): ##for j in range(5): ##velocity[j] = 0.5 *
velocity[j] + 0.5 * (pbest[j] - particles[j]) + 0.5 * (gbest - particles[j]) ##particles[j] = particles[j] + velocity[j]
##if fitness(particles[j]) < fitness(pbest[j]): ##pbest[j] = particles[j] ## gbest = min(pbest, key=fitness) ##
print(i, "\t\t", round(gbest, 2), "\t\t", round(fitness(gbest), 4)) ## print("\nOptimal Position =", round(gbest,
2)) ## print("Minimum Fitness =", round(fitness(gbest), 4))
```

10. Fuzzy Logic Controller – Fan Speed

Algorithm: ## 1. Define temperature as the input variable and fan speed as the output variable. ## 2. Define temperature membership functions as Low, Medium, and High. ## 3. Define fan speed membership functions as Slow, Medium, and Fast. ## 4. Create fuzzy rules relating temperature to fan speed. ## 5. Provide different temperature values and calculate the corresponding fan speed. ## 6. Display the fuzzy control output for each temperature value.

```
## Program: ## import numpy as np ## import skfuzzy as fuzz ## from skfuzzy import control as ctrl ##
temperature = ctrl.Antecedent(np.arange(0, 51, 1), 'temperature') ## fan = ctrl.Consequent(np.arange(0,
101, 1), 'fan') ## temperature['low'] = fuzz.trimf(temperature.universe, [0, 0, 25]) ## temperature['medium']
= fuzz.trimf(temperature.universe, [15, 25, 35]) ## temperature['high'] = fuzz.trimf(temperature.universe,
[25, 50, 50]) ## fan['slow'] = fuzz.trimf(fan.universe, [0, 0, 50]) ## fan['medium'] = fuzz.trimf(fan.universe,
[25, 50, 75]) ## fan['fast'] = fuzz.trimf(fan.universe, [50, 100, 100]) ## rule1 = ctrl.Rule(temperature['low'],
fan['slow']) ## rule2 = ctrl.Rule(temperature['medium'], fan['medium']) ## rule3 =
ctrl.Rule(temperature['high'], fan['fast']) ## system = ctrl.ControlSystem([rule1, rule2, rule3]) ## simulation
= ctrl.ControlSystemSimulation(system) ## print("Temperature\tFan Speed") ## for temp in [15, 25, 35,
45]: ## simulation.input['temperature'] = temp ## simulation.compute() ## print(temp, "\t\t",
round(simulation.output['fan'], 2))
```

11. ANN – $y = 2x + 3$

Algorithm: ## 1. Generate suitable training input values and calculate the corresponding output using $y = 2x + 3$. ## 2. Create an ANN model with an input layer, hidden layer, and output layer. ## 3. Train the ANN using the generated training data. ## 4. Provide a new input value to the trained network. ## 5. Predict the output for the new input. ## 6. Display the predicted output.

```
## Program: ## import numpy as np ## from sklearn.neural_network import MLPRegressor ## X_train =
np.array([[1], [2], [3], [4], [5]]) ## y_train = np.array([5, 7, 9, 11, 13]) ## model =
MLPRegressor(hidden_layer_sizes=(10,), max_iter=5000, random_state=1) ## model.fit(X_train, y_train)
## X_test = np.array([[6]]) ## predicted = model.predict(X_test) ## print("Input:", X_test[0][0]) ##
print("Actual:", 15) ## print("Predicted:", round(predicted[0], 2))
```

12. PID – Motor Speed Control

Algorithm: ## 1. Define the desired motor speed and initialize the actual speed. ## 2. Set the PID controller parameters K_p , K_i , and K_d . ## 3. Calculate the speed error between the desired and actual speed. ## 4. Calculate the proportional, integral, and derivative terms. ## 5. Calculate the PID control signal and update the motor speed. ## 6. Repeat the process for several time steps and display the motor speed and control signal.

```
## Program: ## Kp = 2.0 ## Ki = 0.1 ## Kd = 0.5 ## setpoint = 100 ## speed = 0 ## integral = 0 ##
previous_error = 0 ## print("Time\tSpeed\tControl") ## for t in range(1, 11): ## error = setpoint - speed
## integral = integral + error ## derivative = error - previous_error ## control = Kp * error + Ki *
integral + Kd * derivative ## speed = speed + 0.1 * control ## print(t, "\t", round(speed, 2), "\t",
round(control, 2)) ## previous_error = error
```

13. Adaptive Control – First-Order System

Algorithm: ## 1. Define the desired output and initialize the system output and controller parameter. ## 2. Calculate the tracking error between the desired and actual output. ## 3. Update the controller parameter based on the tracking error. ## 4. Calculate the control action using the updated parameter. ## 5. Update the system output and repeat the process for several time steps. ## 6. Display the output, error, and final controller parameter.

```
## Program: ## setpoint = 10 ## output = 0 ## K = 0.5 ## learning_rate = 0.01 ##
print("Time\tOutput\tError\tGain") ## for t in range(1, 11): ##     error = setpoint - output ##     K = K +
learning_rate * abs(error) ##     output = output + K * error * 0.1 ##     print(t, "\t", round(output, 2), "\t",
round(error, 2), "\t", round(K, 2))
```

14. Genetic Algorithm – Minimum of $(x - 3)^2 + 1$

Algorithm: ## 1. Define the objective function $f(x) = (x - 3)^2 + 1$ and specify the search range. ## 2. Randomly initialize a candidate solution within the specified range. ## 3. Calculate the fitness of the candidate solution. ## 4. Generate a new candidate using mutation. ## 5. Select the candidate with the better fitness value. ## 6. Repeat the process for several generations and display the best solution and minimum value.

```
## Program: ## import random ## def fitness(x): ##     return (x - 3) ** 2 + 1 ## x = random.uniform(0, 10)
## print("Generation\tX\tFitness") ## for generation in range(1, 11): ##     new_x = x +
random.uniform(-0.5, 0.5) ##     if 0 <= new_x <= 10 and fitness(new_x) < fitness(x): ##         x = new_x
##     print(generation, "\t\t", round(x, 2), "\t", round(fitness(x), 4)) ## print("\nBest Solution =", round(x, 2))
## print("Minimum Value =", round(fitness(x), 4))
```

15. Q-Learning – 4×4 Grid with Obstacles

Algorithm: ## 1. Define the 4×4 grid with the start position, goal position, and obstacles. ## 2. Initialize the Q-table with zeros and set the discount factor. ## 3. Start the agent from the initial state and select valid actions randomly during training. ## 4. Update the Q-value using the reward and maximum future Q-value. ## 5. Continue training until the agent learns to reach the goal while avoiding obstacles. ## 6. Display the learned Q-table and the optimal path from start to goal.

```
## Program: ## import numpy as np ## import random ## R = np.array([[ -1, 0, -1, -1], [0, -1, 0, -1], [-1, 0,
-1, 0], [-1, -1, 0, 100]]) ## Q = np.zeros((4, 4)) ## gamma = 0.8 ## for episode in range(100): ##     state =
0 ##     while state != 3: ##         actions = np.where(R[state] >= 0)[0] ##         action =
random.choice(actions) ##         Q[state, action] = R[state, action] + gamma * np.max(Q[action]) ##
state = action ## print("Q Table:") ## print(np.round(Q, 2)) ## state = 0 ## print("\nOptimal Path:") ##
while state != 3: ##     print(state, end=" -> ") ##     state = np.argmax(Q[state]) ## print(3)
```

16. MPC – Position Control

Algorithm: ## 1. Define the desired position and initialize the current position of the moving object. ## 2. Calculate the position error between the desired and current positions. ## 3. Calculate the MPC control action based on the current error. ## 4. Apply the control action and update the object position. ## 5. Repeat the prediction and control process for every time step. ## 6. Display the position and control signal until the desired position is reached.

```
## Program: ## setpoint = 10 ## position = 0 ## print("Step\tPosition\tControl") ## for step in range(1, 11):
##     error = setpoint - position ##     control = 0.5 * error ##     position = position + control ##
print(step, "\t", round(position, 2), "\t\t", round(control, 2))
```

17. Dijkstra – 5×5 Grid with Obstacles

Algorithm: ## 1. Represent the 5×5 grid as a graph and define free cells and obstacle cells. ## 2. Define the start and goal positions and initialize all distances as infinity. ## 3. Set the distance of the start node to zero and insert it into the priority queue. ## 4. Select the node with the smallest distance and examine its valid neighboring cells. ## 5. Update the distance of each neighbor if a shorter path is found. ## 6. Continue until the goal is reached and display the shortest path and its cost.

```
## Program: ## import heapq ## graph = { ##     'A': {'B': 1, 'F': 1}, ##     'B': {'A': 1, 'C': 1}, ##     'C': {'B': 1,
'D': 1}, ##     'D': {'C': 1, 'E': 1}, ##     'E': {'D': 1, 'J': 1}, ##     'F': {'A': 1, 'G': 1}, ##     'G': {'F': 1, 'H': 1}, ##
'H': {'G': 1, 'I': 1}, ##     'I': {'H': 1, 'J': 1}, ##     'J': {'E': 1, 'I': 1} ## } ## def dijkstra(graph, start): ##
distance = {node: float('inf') for node in graph} ## distance[start] = 0 ## pq = [(0, start)] ## while
pq: ##     dist, node = heapq.heappop(pq) ##     for neighbor, cost in graph[node].items(): ##
new_dist = dist + cost ##     if new_dist < distance[neighbor]: ##         distance[neighbor] =
new_dist ##     heapq.heappush(pq, (new_dist, neighbor)) ## return distance ## result =
dijkstra(graph, 'A') ## print("Shortest Distance from A") ## for node, dist in result.items(): ##     print(node,
```

"=", dist)

18. PSO – Maximum of $f(x) = 8 - (x - 2)^2$

Algorithm: ## 1. Define the objective function $f(x) = 8 - (x - 2)^2$. ## 2. Initialize particles with random positions and velocities within the search space. ## 3. Evaluate the fitness of each particle and store its personal best position. ## 4. Determine the global best position by selecting the particle with the maximum fitness. ## 5. Update particle velocities and positions using personal best and global best values. ## 6. Repeat the process for several iterations and display the optimal position and maximum fitness value.

```
## Program: ## import random ## def fitness(x): ##     return 8 - (x - 2) ** 2 ## particles =  
[random.uniform(0, 5) for _ in range(5)] ## velocity = [0] * 5 ## pbest = particles[:] ## gbest = max(pbest,  
key=fitness) ## print("Iteration\tGlobal Best\tFitness") ## for i in range(1, 11): ##     for j in range(5): ##  
velocity[j] = 0.5 * velocity[j] + 0.5 * (pbest[j] - particles[j]) + 0.5 * (gbest - particles[j]) ##         particles[j] =  
particles[j] + velocity[j] ##         if fitness(particles[j]) > fitness(pbest[j]): ##             pbest[j] = particles[j] ##  
gbest = max(pbest, key=fitness) ##     print(i, "\t\t", round(gbest, 2), "\t\t", round(fitness(gbest), 4)) ##  
print("\nOptimal Position =", round(gbest, 2)) ## print("Maximum Fitness =", round(fitness(gbest), 4))
```